TECHNICAL DESIGN REPORT

Crypto16 Coprocessor

A 16-Bit Register-Based Hardware Datapath for Cryptographic Operations

Team and Task Distribution

No.	Name	Academic No.	Task completed
1	Wateen Walid Mohamed Baree	2951	Register file / Report
2	Ibraheem Shaaban Abdel Nabi	2901	LUT / Report
3	Ahmed Tharwat Salah	2902	Combinational logic integration / Presentation
4	Ahmed Salah Abdel Majeed	2903	Shifter / Schematic
5	Ahmed Abdel Fattah Elmallah	2904	ALU / Schematic
6	Ahmed Mahmoud Abdel Fattah	2905	Coprocessor integration (including **control unit**) / Presentation

**Task distribution is done by Ahmed Mahmoud Abdel Fattah

ABSTRACT

This report presents a comprehensive technical analysis of the Crypto16 Coprocessor — a compact 16-bit hardware design programmed in **VHDL**. The system is built to handle three specific tasks: arithmetic and logic operations (ALU), fixed-pattern shift and rotation operations (Shifter), and nonlinear byte substitution via an 8-bit lookup table (LUT). The architecture follows a two-phase cycle model with asynchronous register reads and synchronous write-back, enabling deterministic operation sequencing. This document covers the hardware interfaces, the internal setup, instruction coding, and the testing methods used to prove the design works correctly.

Table of Contents

1. Introduction and Design Motivation
2. System Architecture Overview
3. Top-Level Entity: crypto16_coprocessor
4. Register File Subsystem
5. Control unit
6. Arithmetic and Logic Unit (ALU)
7. Shifter Block
8. Nonlinear LUT Substitution Block
9. Combinational Result Selection Logic
10. Timing Model and Pipeline Behavior
11. Verification and Testbench Strategy
12. Design Strengths
13. References

1. Introduction and Design Motivation

The **Crypto16 Coprocessor** is a 16-bit hardware datapath designed as a practical learning tool for digital logic and embedded systems. Its main goal is to show how a single, organized unit can handle three essential types of operations: math operations (ALU), data shifting (Shifter), and data substitution (LUT). The Crypto16 simplifies these complex concepts into a 16-bit model, making it easier to understand how hardware security actually works. This design serves as a bridge between basic digital logic and the high-level architecture used in professional cryptographic hardware.

The design is implemented entirely in **VHDL**, using a mix of structural and behavioral coding. The code is written to be clear and easy to test. Each functional block is built as an independent unit with its own defined ports. This modular approach makes it easy to test each part individually before simulating the entire system together.

The architecture prioritizes the following design objectives:

- **Modularity:** Clean separation between storage, computation, and control
- **Educational clarity:** Explicit ripple-carry adder structure, readable LUT tables, and commented opcode constants
- **All-in-One Design:** A single execution path handles all three essential types of encryption operations (Math operations, Shifting, and Substitution).

2. System Architecture Overview

The **Crypto16 Coprocessor** is organized around a straightforward datapath model in which operands are sourced from a central register file, processed by one of three execution engines, and written back to the register file. The following block diagram conceptually illustrates this organization:

Crypto16 Coprocessor — Functional Block Summary		
Block / Entity	**Function**	**Type**
crypto16_coprocessor	Top-level integration and control	Structural
register_file	16 × 16-bit register storage	RTL
combinational	Main datapath integrator that connects the control unit, execution engines, and output Mux.	Structural & Behavioral
control_unit	Decodes the 4-bit control signal into a sel_block & operation-specific control signals for ALU and Shifter	Behavioral
ALU	ADD, SUB, AND, OR, XOR, NOT, MOVE	Combinational
shifter	ROR8, ROR4, SLL8 operations	Combinational
NonLinearLut	8-bit nibble substitution (S-Box)	Combinational
Adder_16bit	16-bit ripple-carry adder	Structural
Full_Adder	Single-bit full adder primitive	Structural

Crypto16 Coprocessor entity hierarchy and functional classification.

The **combinational logic** includes a control unit that acts as a selector by decoding the 4-bit control signal into internal selection signals `sel_block`. These signals determine whether the ALU, Shifter, or LUT is activated. The selected result is then forwarded to the register file for storage. This approach follows the standard register-based datapath design found in most digital logic textbooks.

3. Top-Level Entity: crypto16_coprocessor

3.1 Port Interface

The top-level entity exposes a minimal, clean interface that encapsulates all external communication requirements of the coprocessor:

Signal	Direction	Width	Description
Ra	Input	4 bits	Source register A index (0–15)
Rb	Input	4 bits	Source register B index (0–15)
Rd	Input	4 bits	Destination register index (0–15)
ctrl	Input	4 bits	Operation control code
clk	Input	1 bit	System clock (Rising-edge triggered)
res	Input	1 bit	Hybrid active-high reset

Top-level port interface for crypto16_coprocessor.

3.2 Internal Control Logic

The top-level architecture registers `ctrl` and `Rd` into `ctrl_temp` and `Rd_temp` on each rising clock edge. The write-enable signal `wrt` is asserted for all control codes except `0111`, which is designated as a write-disable (NOP-style) control. Importantly, the `wrt` signal is evaluated directly from the `ctrl`, ensuring that the write-disable condition `0111` is applied immediately within the same clock cycle.

The reset mechanism is implemented using a **hybrid reset synchronizer** `res_hybrid`, which combines asynchronous assertion with synchronous deassertion. This ensures that reset is immediately activated when `res = '1'`, while deassertion is safely synchronized to the clock to avoid **metastability** issues. When `res_hybrid` is active, all internal registers (`ctrl_temp`, `Rd_temp`) are cleared and `wrt` is forced to '0'. During normal operation, control and address signals are updated on the rising edge of the clock.

4. Register File Subsystem

The **register_file** entity implements the architectural register state of the Crypto16 Coprocessor. It provides storage for 16 general-purpose 16-bit registers and supports concurrent asynchronous reads on two ports alongside a single synchronous write port.

4.1 Storage Model

Storage is defined as an internal array: `reg_data : array(0 to 15) of std_logic_vector(15 downto 0)`. An asynchronous reset (`res = '1'`) immediately clears all 16 registers, ensuring that the design returns to a deterministic all-zero state regardless of clock phase.

4.2 Read and Write Paths

The **asynchronous read** paths continuously drive `ABus` and `BBus` from the register entries indexed by `Ra` and `Rb` respectively. This eliminates any clock-to-data latency on the read side, reducing the critical path length for operand availability. The **synchronous write** path commits the `Result` bus to register `registers[Rd]` on the rising edge of `clk` only when `wrt = '1'`. This combination — asynchronous read / synchronous write — is the canonical design pattern for small register files in academic HDL implementations.

4.3 Reset Behavior

The reset is implemented as an asynchronous signal within the process. When `res` is asserted, all registers clear to `x"0000"` without waiting for a clock edge. This behavior is consistent with flip-flop-level reset semantics as described in standard VHDL textbooks.

5. Control Unit

The **control_unit** is responsible for decoding the 4-bit input control signal `ctrl` into a set of internal control signals that govern the datapath operation. It acts as the decision-making component that determines which execution block is activated and what specific operation is performed within that block.

5.1 Function

The control unit translates the external 4-bit opcode into three internal signals:

- `sel_block`: A 2-bit signal that selects which execution unit (ALU, Shifter, or LUT) drives the final result
- `alu_op`: A 3-bit signal that specifies the operation to be performed by the ALU
- `shift_op`: A 2-bit signal that specifies the operation to be performed by the Shifter

This decoding enables a clean separation between instruction encoding and datapath execution.

5.2 Block Selection

Ctrl Range	Sel_block	Selected Block
0000-0111	00	ALU - ctrl[2:0] → alu_op
1000-1010	01	Shifter - ctrl[1:0] → shift_op
1011	10	NonLinearLut - direct selection
1100-1111	11	No operation (NOP) - no output

Block selection mapping generated by the control unit

6. Arithmetic and Logic Unit (ALU)

The **ALU** implements the core data transformation operations required for general-purpose register manipulation. It is a fully combinational block whose output is determined by ABus, BBus, and a **3-bit** operation control signal derived from the control unit.

6.1 Supported Operations

ALU op code	Operation	Description
000	ADD	16-bit addition: A + B using ripple-carry adder
001	SUB	16-bit subtraction: A − B via two's complement (A + ~B + 1)
010	AND	Bitwise logical AND: A AND B
011	OR	Bitwise logical OR: A OR B
100	XOR	Bitwise logical XOR: A XOR B
101	NOT	Bitwise complement of A (B ignored)
110	MOVE	Pass-through: Output = A (register copy)

ALU supported operations and control codes.

6.2 Adder Architecture

Addition and subtraction are both handled by the **Adder_16bit** module, which is constructed from 16 instances of the **Full_Adder** primitive in a ripple-carry topology. The carry chain propagates from `carry(0) = cin` to `carry(16) = cout`, implementing a structurally explicit design that is highly readable in simulation. The full adder sum and carry-out logic is:

`Sum =a XOR b XOR cin`; `cout = (a AND b)OR(b AND cin)OR(cin AND a)`

For subtraction, the `BBus` input is bitwise inverted and the carry-in is forced to `'1'`, realizing two's complement negation in accordance with standard binary arithmetic principles. While the ripple-carry topology is not optimal for timing in deep pipelines, it is chosen here for its structural clarity and ease of formal reasoning .

6.3 Fallback Behavior

For any unsupported 3-bit ALU control value, the output is zeroed: `Result <= (others => '0')`. This prevents undefined behavior and supports deterministic simulation.

7. Shifter Block

The **Shifter** block implements three fixed-pattern positional transformation operations on the 16-bit `ABus` value. Shifter operations are selected via a 2-bit control signal `shift_Ctrl` generated by the control unit.

7.1 Supported Operations

shift_Ctrl	Operation	Behavior
00	ROR 8	Rotate right by 8 bits — upper and lower bytes swapped
01	ROR 4	Rotate right by 4 bits — nibble rotation
10	SLL 8	Logical shift left by 8 bits — lower byte zeroed, upper byte = original lower
others	NOP (zero)	All other codes produce 16'h0000

Shifter block operations.

The shifter employs `unsigned` type conversions from `ieee.numeric_std` to apply the `ror` and `sll` operators, then casts the result back to `std_logic_vector`. This approach avoids overloading ambiguity and is consistent with recommended VHDL numeric coding practices. Shift and rotate operations are fundamental to many lightweight block cipher structures, including SIMON and SPECK, and the presence of byte-granularity rotation in the Crypto16 design makes it directly applicable to such primitives.

8. Nonlinear LUT Substitution Block

The **NonlinearLut** entity implements an 8-bit nonlinear substitution operation analogous to the S-Box constructions used in modern symmetric cryptographic algorithms. S-Box nonlinearity is the primary defense against linear and differential cryptanalysis.

8.1 Operational Principle

The lower 8 bits of `ABus` are presented to the LUT block as `LutIn(7:0)`. These are split into upper nibble (`S_Box1 = LutIn(7:4)`) and lower nibble (`S_Box2 = LutIn(3:0)`). Each nibble independently undergoes a

4-bit → 4-bit substitution, and the results are concatenated to form the transformed 8-bit output `LutOut`. The upper byte of `ABus` passes through unchanged, producing the final 16-bit result: `Result <= ABUS(15 downto 8) & LUT_out_temp`.

8.2 Substitution Tables

The two nibble substitution tables are defined below. These form a pair of 4-bit bijective mappings (permutations over GF(2^4)):

S_Box1 (Upper Nibble) and S_Box2 (Lower Nibble) Substitution Tables										
Input	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
S1 Out	1	B	9	C	D	6	F	3	E	8
S2 Out	F	0	D	7	B	E	5	A	9	2
Input	**A**	**B**	**C**	**D**	**E**	**F**	**—**	**—**	**—**	**—**
S1 Out	7	4	A	2	5	0	—	—	—	—
S2 Out	C	1	3	4	8	6	—	—	—	—

Nibble substitution (S-Box) tables for the NonlinearLut block.

The bijective nature of each 4-bit substitution guarantees that the transformation is invertible — a necessary property for use in symmetric encryption and decryption contexts. The combined nonlinearity of both S-Boxes over the 8-bit input space contributes resistance to linear approximation attacks.

9. Combinational Result Selection Logic

The **combinational** entity (`combinational.vhd`) is responsible for instantiating the ALU, Shifter, and NonlinearLut blocks, distributing operands, and multiplexing the results based on the decoded selection signal `sel_block` generated by the control unit.

9.1 Result Source Multiplexing

sel_block	Selected Source	Notes
00	ALU Output	Standard arithmetic/logic;
01	Shifter Output	Positional data transformations
10	LUT Output	Nonlinear byte substitution
others	x"0000"	Reserved; unused codes zero the output

Combinational result source selection controlled by sel_block .

9.2 Library Note

The combinational module itself does not perform arithmetic interpretation of the control signal; instead, it relies on pre-decoded control outputs generated by the control unit. All arithmetic operations within the system are implemented using `ieee.numeric_std` for portability.

10. Timing Model and Pipeline Behavior

The Crypto16 Coprocessor is a **single-cycle datapath** with registered control signals and a synchronous write-back stage. Although it is not a pipelined architecture, its operation can be described in terms of two functional phases to clarify timing behavior.

Phase A — Combinational Evaluation

Between rising clock edges, operand values are read asynchronously from the register file through ABus and BBus. The selected execution unit (ALU, Shifter, or NonlinearLut) computes the `Result` combinationally based on the decoded control signal `sel_block`. This phase represents the complete datapath evaluation stage where all arithmetic, logical, or substitution operations are resolved before the clock edge.

Phase B — Sequential Commit

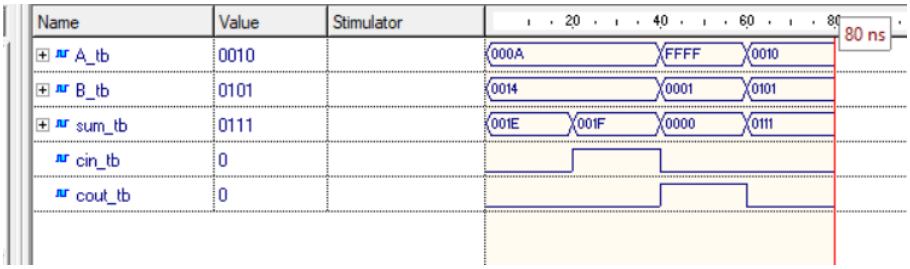
On the rising edge of `clk`, if `wrt = '1'`, the computed `Result` is written into the destination registers `registers[Rd]`, and at the same time `ctrl` and `Rd` are sampled into internal registers (`ctrl_temp` and `Rd_temp`). This introduces a registered control path, where control signals are sampled at the clock edge before being used in the combinational datapath, ensuring that control decisions are applied in a synchronized manner across clock cycles.

11. Test Bench Strategy

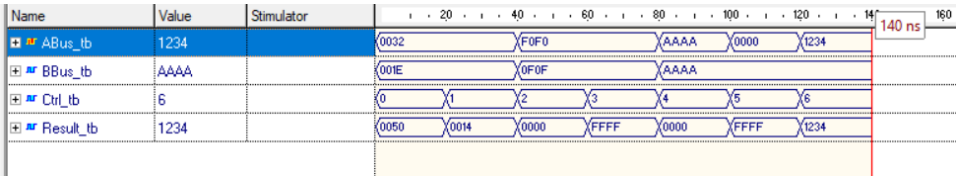
The Crypto16 Coprocessor design is accompanied by a structured suite of directed-stimulus testbenches covering both unit-level and integration-level validation. All testbenches are located in the sim/ directory and are intended for simulation in standard VHDL simulation environments (ModelSim, GHDL, Vivado Simulator).

11.1 Unit-Level Testbenches

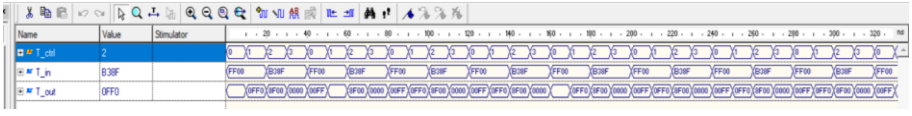
- **Adder_16bit_TB.vhd:**
  validates carry propagation, overflow scenarios, and corner cases (0+0, 0xFFFF+1, etc.)



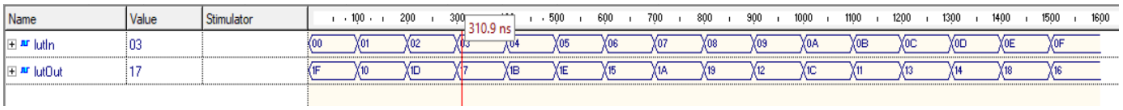
- **ALU_TB.vhd:**
  directed tests for all seven ALU operations including edge values



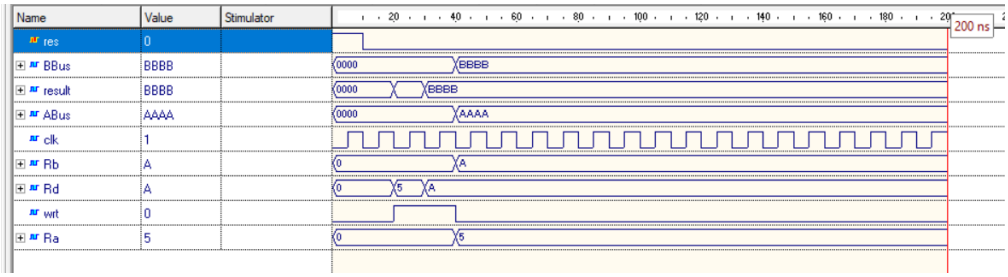
- **shifter_TB.vhd:**
  validates all three shift/rotate operations and the zero-output fallback for undefined codes



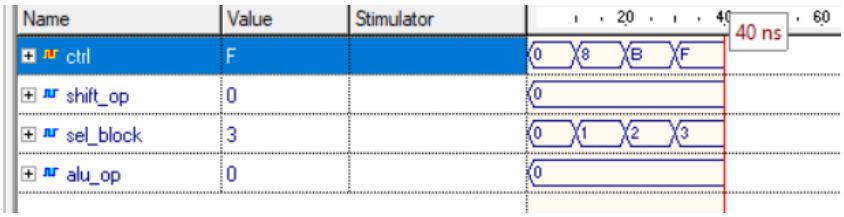
- **NonLinearLut_TB.vhd:**
  sweeps all 256 possible 8-bit input values through the LUT substitution



- **register_file_TB.vhd :**
  verifies asynchronous reset, write-enable gating, and read-back integrity

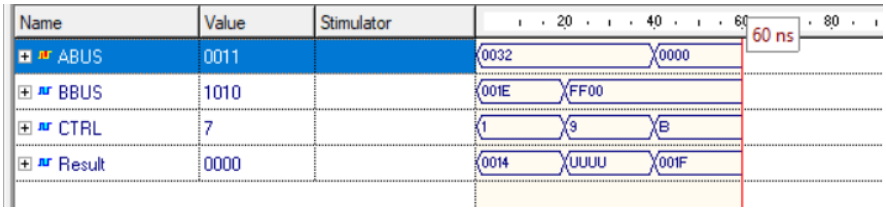


- **control_unit_TB.vhd:**
  exhaustively sweeps all 16 control inputs to validate block selection boundaries and opcode propagation.

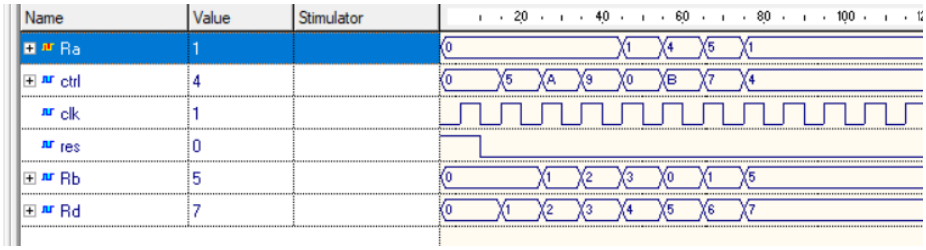


11.2 Integration-Level Testbenches

- **combinational_TB.vhd:**
  validates result source multiplexing by driving the control unit inputs across all execution ranges.



- **crypto16_coprocessor_TB.vhd:**
  end-to-end integration test: sequences register writes, operation execution, and readback through the full top-level interface



11.3 Verification Methodology Notes

All testbenches employ directed stimulus with expected outcomes documented in inline comments and assertion report messages. This approach is appropriate for initial functional bring-up and waveform inspection. It does not include constrained-random stimulus or formal coverage analysis.

12. Design Strengths

The Crypto16 Coprocessor exhibits a set of strong architectural characteristics that make it suitable as a modular and educational hardware datapath design. These characteristics highlight the clarity, structure, and extensibility of the system implementation.

12.1 Strengths

- **Highly modular:** each functional block is independently instantiable and testable
- **Structurally explicit arithmetic:** ripple-carry adder allows direct gate-level inspection and formal verification
- **Clear separation of concerns:** storage, computation, and control are logically partitioned
- **Deterministic reset behavior:** all state clears to well-defined zero values
- **Supports the three** main classes of cryptographic operations within a single integrated datapath.

13. References

[1] D. R. Stinson and M. B. Paterson, Cryptography: Theory and Practice, 4th ed. Boca Raton, FL: CRC Press/Taylor & Francis, 2019.

[2] C. Paar and J. Pelzl, Understanding Cryptography: A Textbook for Students and Practitioners, Berlin, Germany: Springer-Verlag, 2010.

[3] F. Vahid and T. Givargis, Embedded System Design: A Unified Hardware/Software Introduction, New York, NY: John Wiley & Sons, 2002.

[4] P. J. Ashenden, The Designer's Guide to VHDL, 3rd ed. Burlington, MA: Morgan Kaufmann, 2008.

[5] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: Hardware/Software Interface (RISC-V Edition), 2nd ed. San Francisco, CA: Morgan Kaufmann, 2020.

[6] IEEE Computer Society, IEEE Standard VHDL Language Reference Manual, IEEE Std 1076-2008. New York, NY: IEEE, 2009.

[7] S. Brown and Z. Vranesic, Fundamentals of Digital Logic with VHDL Design, 3rd ed. New York, NY: McGraw-Hill, 2009.

[8] J. Bhasker, A VHDL Primer, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 1999.

[9] R. L. Tokheim, Digital Electronics: Principles and Applications, 8th ed. New York, NY: McGraw-Hill Education, 2016.

[10] M. M. Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed. Hoboken, NJ: Pearson, 2018.

[11] N. H. E. Weste and D. Harris, CMOS VLSI Design: A Circuits and Systems Perspective, 4th ed. Boston, MA: Addison-Wesley, 2011.

[12] P. J. Ashenden and J. Lewis, VHDL-2008: Just the New Stuff, Burlington, MA: Morgan Kaufmann, 2008.

[13] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "SIMON and SPECK: Block Ciphers for the Internet of Things," IACR Cryptology ePrint Archive, Report 2013/404, 2013. [Online]. Available: https://eprint.iacr.org/2013/404

[14] J. Daemen and V. Rijmen, The Design of Rijndael: AES — The Advanced Encryption Standard, Berlin, Germany: Springer-Verlag, 2002.

[15] M. Matsui, "Linear Cryptanalysis Method for DES Cipher," in Advances in Cryptology — EUROCRYPT '93, Lecture Notes in Computer Science, vol. 765, T. Helleseth, Ed. Berlin, Germany: Springer, 1994, pp. 386–397.

[16] E. Biham and A. Shamir, Differential Cryptanalysis of the Data Encryption Standard, New York, NY: Springer-Verlag, 1993.

[17] D. A. Patterson and J. L. Hennessy, Computer Architecture: A Quantitative Approach, 6th ed. Waltham, MA: Morgan Kaufmann, 2017.

[18] J. Bergeron, Writing Testbenches: Functional Verification of HDL Models, 2nd ed. Norwell, MA: Kluwer Academic Publishers, 2003.

— END OF REPORT —